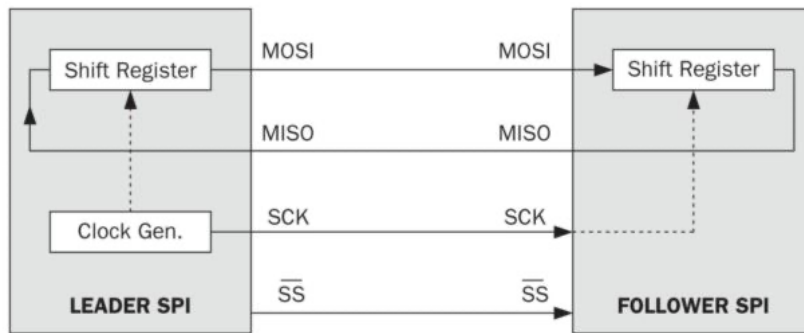


Week 10 Serial Peripheral Interface (SPI)

1. Basics

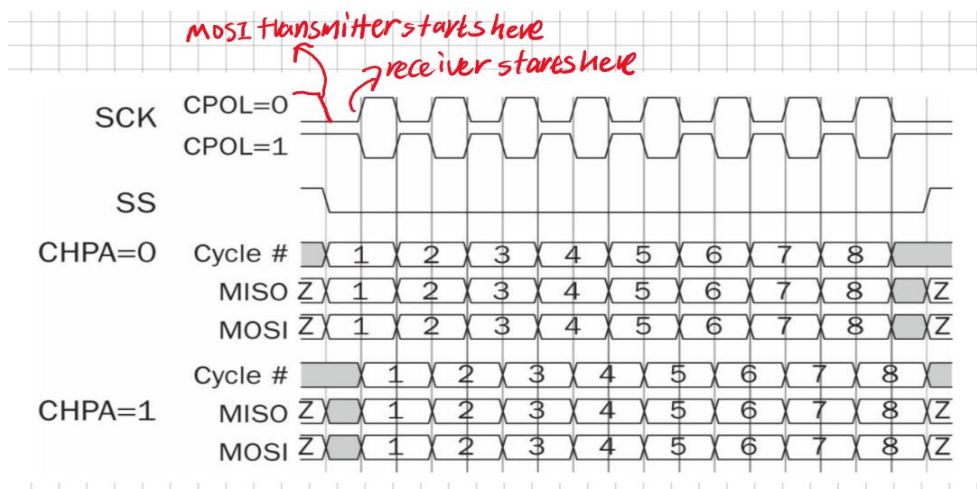
Different from the UART, the SPI is a synchronous communication protocol. Besides, communication between the transmitter and receiver is duplex. In other words, data is transmitted and received at the same time in the SPI. Therefore, the SPI communication uses four wires. Two of these wires are for data transfer. One wire is used for the common clock signal (for synchronization). The fourth wire is used to enable (select) signal

Different from the UART, data packet size is not constant in the SPI. This is an advantage since the user can select the packet size as he or she desires. Moreover, the dedicated common clock and enable signals avoid using start and stop bits in the UART. The only requirement here is the need for determining the data packet size. Hence, the transmitter and receiver can understand each other.



SCK is generated by the leader and fed to the follower

Here, SCK depends on the maximum data rate of the transmitter and receiver. Hence, the device with the lowest rate defines its limit. Besides frequency, the leader also adjusts the polarity and phase of clock denoted by CPOL and CPHA, respectively. Four possible combinations of CPOL and CPHA are presented in the following figure. These combinations are called modes of the SPI.



Transmission and Reception Operations

In the SPI, the data transmission and reception is controlled by the leader through SCK and signals. When there is no transmission, stays at logic level 1 and SCK stays either at logic level 0 or 1 depending on the SPI mode.

The SPI communication starts when the leader wakes the follower by setting to logic level 0. Next, the leader and follower start interchanging data in every clock cycle set by SCK. Here, either the leader sends a bit through MOSI line or the follower sends a bit through MISO line. The SPI mode also determines if data will be sent on the rising or falling edge of SCK. After all bits are transferred, the common clock stops and leader deselects the follower by changing to logic level 1.

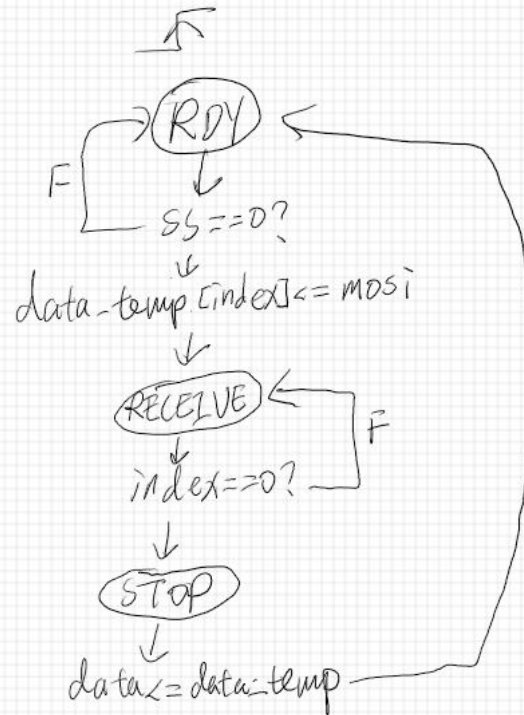
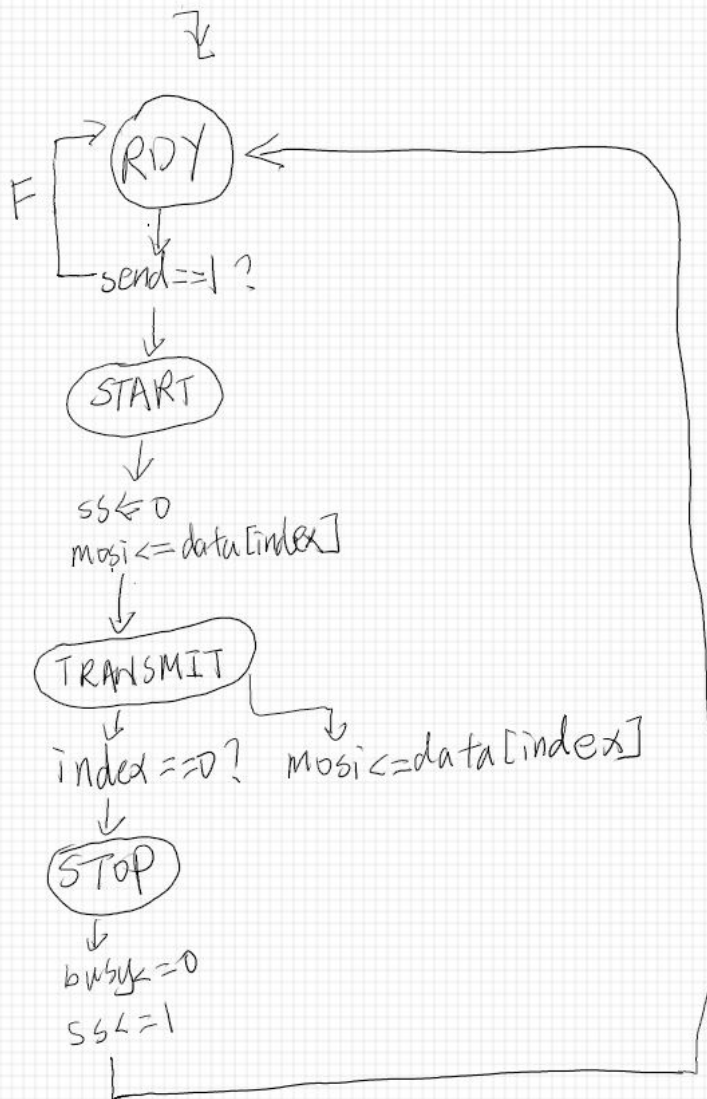
Modes		Clock base	Data availability	Bits on MISO and MOSI lines		
CPOL	CPHA			Placed	Stay	Captured
0	0	0	0 to 1 transition	Clock goes to 0	Clock at 1	0 to 1 transition
0	1	1	0 to 1 transition	Clock goes to 1	Clock at 0	1 to 0 transition
1	0	0	1 to 0 transition	Clock goes to 1	Clock at 0	1 to 0 transition
1	1	1	1 to 0 transition	Clock goes to 0	Clock at 1	0 to 1 transition

Read [this PDF](#) from the textbook to understand the operation of SPI.

MOSI:

The MOSI of the leader and the follower:

Leader transmitter:



Leader transmitter and follower receiver:

The image shows a Verilog code editor with two modules: `SPI_leader_transmitter` and `SPI_follower_receiver`. Handwritten annotations in red ink provide timing and state information.

Left Module: `SPI_leader_transmitter`

```

1 module SPI_leader_transmitter(clk,data,send,sck,ss,mosi,busy);
2 parameter data_length=8;
3 input clk;
4 input [data_length-1:0] data;
5 input send;
6 output reg sck=0;
7 output reg ss=1;
8 output reg mosi;
9 output reg busy=0;
10 localparam RDY=2'b00,START=2'b01,TRANSMIT=2'b10,STOP=2'b11;
11 reg [1:0] state=RDY;
12 reg [7:0] clkdiv=0;
13 reg [7:0] index=0;
14 always @(posedge clk)
15 //sck is set to 2MHz
16 if (clkdiv == 8'd25)
17 begin
18     clkdiv<=0;
19     sck<=~sck;
20 end
21 else clkdiv<=clkdiv + 1;
22
23 always @(negedge sck)
24 case (state)
25 RDY:
26     if(send)
27     begin
28         state<= START;
29         busy<=1;
30         index <= data_length -1;
31     end
32 START:
33     begin
34         ss<=0;
35         mosi<=data[index]; // assume data is already
36                             // there
37         index<=index-1;
38         state<=TRANSMIT;
39     end
40 TRANSMIT:
41     begin
42         if(index==0)
43             state<= STOP;
44         mosi<=data[index];
45         index<=index-1;
46     end
47 STOP:
48     begin
49         busy<=0;
50         ss<=1;
51         state<=RDY;
52     end
53 endcase
54 endmodule

```

Handwritten Annotations for Transmitter:

- 2 MHz
- $\frac{100\text{MHz}}{2\text{MHz}} = 50$
- 50 cycles
- The 25th rising edge
- // assume data is already there

Right Module: `SPI_follower_receiver`

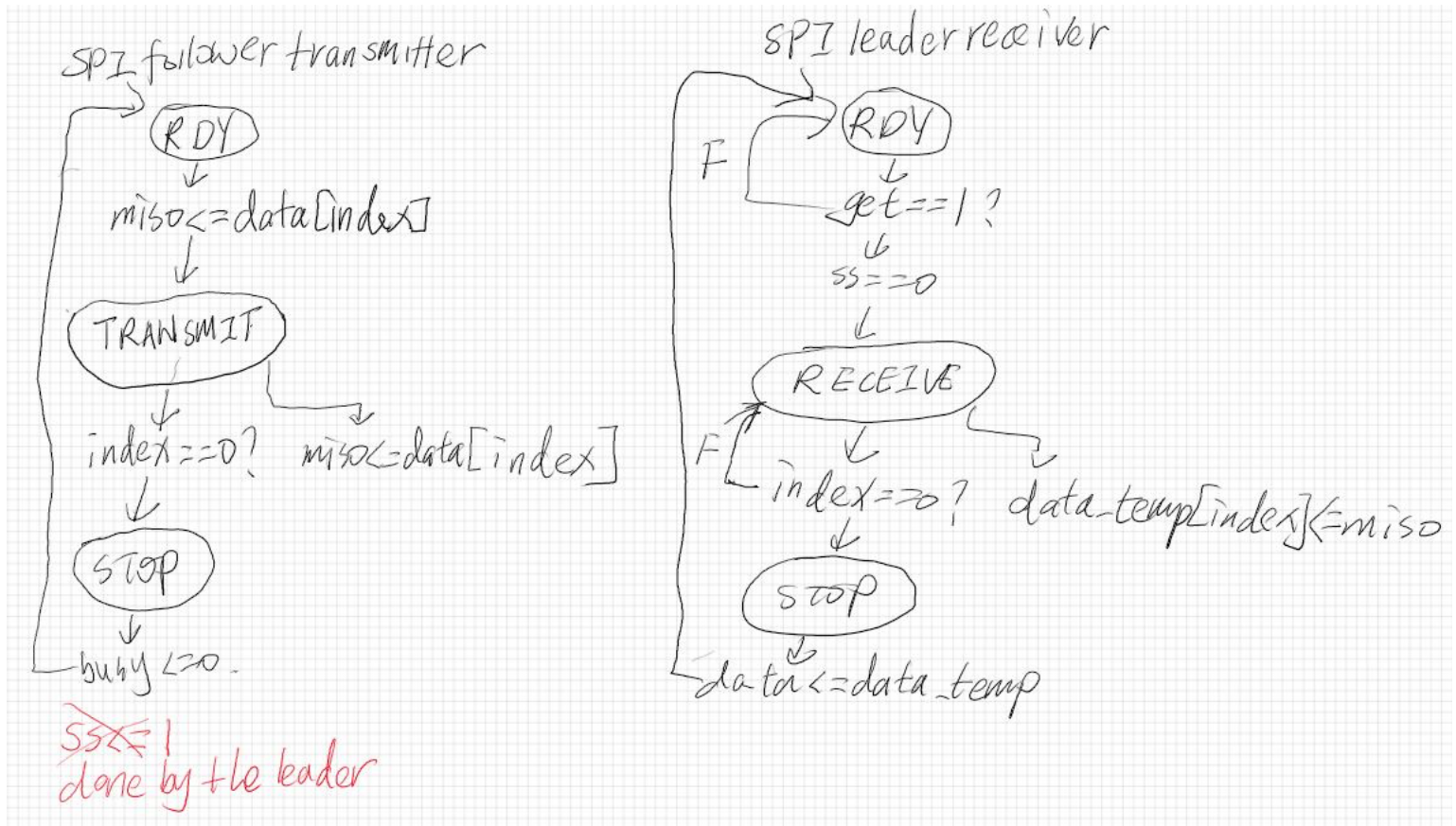
```

1 module SPI_follower_receiver(sck,ss,mosi,data,busy,ready);
2 parameter data_length=8;
3 input sck;
4 input ss;
5 input mosi;
6 output reg [data_length-1:0] data;
7 output reg busy=0;
8 output reg ready=0;
9 localparam RDY=2'b00,RECEIVE=2'b01,STOP=2'b10;
10 reg [1:0] state=RDY;
11 reg [data_length-1:0] data_temp=0;
12 reg [7:0] index=data_length-1;
13
14 always @(posedge sck)
15 case(state)
16 RDY:
17     if(!ss)
18     begin
19         data_temp[index]<=mosi;
20         index<=index-1;
21         busy<=1;
22         ready<=0;
23         state<=RECEIVE;
24     end
25 RECEIVE:
26     begin
27         if(index==0)
28             state<=STOP;
29         else index<=index-1;
30         data_temp[index] <= mosi;
31     end
32 STOP:
33     begin
34         busy<=0;
35         ready<=1;
36         data_temp<=0;
37         data<=data_temp;
38         index<=data_length-1;
39         state<=RDY;
40     end
41 endcase
42 endmodule

```

MISO:

The MISO of the leader and the follower:

**Follower transmitter and leader receiver:**

Line 18 for the SPI_leader_receiver module, I used 8'd25 instead of 24 in the textbook.


```

SPL_follower_transmitter.v (~\Documents\courses\0_CE433\lectures\week10_SPI\Verilog) - GVIM
File Edit Tools Syntax Buffers Window Help

1 module SPI_leader_receiver(clk,miso,get,data,sck,ss,busy,ready);
2 parameter data_length=8;
3 input clk;
4 input get;
5 input miso;
6 output reg [data_length-1:0] data;
7 output reg ss=1;
8 output reg sck=0;
9 output reg busy=0;
10 output reg ready=0;
11 localparam RDY=2'b00,RECEIVE=2'b01,STOP=2'b10;
12 reg [1:0] state=RDY;
13 reg [data_length-1:0] data_temp=0;
14 reg [7:0] index=0;
15 reg [7:0] clkdiv=0;
16
17 always @ (posedge clk)
18     if(clkdiv==8'd25)
19         begin
20             clkdiv<=0;
21             sck<=~sck;
22         end
23     else
24         clkdiv<=clkdiv+1;
25
26 always @ (posedge sck)
27 case(state)
28 RDY:
29     if(get)
30     begin
31         ss<=0;
32         state<=RECEIVE;
33         busy<=1;
34         ready<=0;
35         data_temp<=0;
36         index<=data_length-1;
37     end
38 RECEIVE:
39     begin
40         if(index==0)
41             state<=STOP;
42         data_temp[index]<=miso;
43         index<=index-1;
44     end
45 STOP:
46     begin
47         busy<=0;
48         ready<=1;
49         ss<=1;
50         data_temp<=0;
51         data<=data_temp;
52         state<=RDY;
53     end
54 endcase
55 endmodule

1 module SPI_follower_transmitter(miso,data,sck,ss,busy);
2 parameter data_length=8;
3 input sck;
4 input ss;
5 output reg miso;
6 output reg busy;
7 output [data_length-1:0] data;
8 localparam RDY=2'b00,TRANSMIT=2'b01,STOP=2'b10;
9 reg [1:0] state=RDY;
10 reg [7:0] index=data_length-1;
11
12 always @ (negedge sck)
13 case(state)
14 RDY:
15     if(!ss)
16     begin
17         miso<=data[index];
18         state<=TRANSMIT;
19         busy<=1;
20         index<=index-1;
21     end
22 TRANSMIT:
23     begin
24         begin
25             if(index==0)
26                 state<=STOP;
27             miso<=data[index];
28             index<=index-1;
29         end
30     end
31 STOP:
32     begin
33         index<=data_length-1;
34         busy<=0;
35         state<=RDY;
36     end
37 endcase
38 endmodule

```

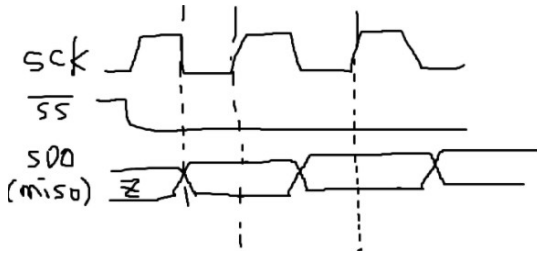
2. SPI Application - the PmodALS ambient light sensor

The Pmod ALS (Ambient Light Sensor) by Digilent is named based on its functionality and form factor: "Pmod" stands for Peripheral Module, an open-standard interface designed by Digilent for connecting small I/O modules to FPGA/microcontroller boards

The PmodALS communicates with the host board via the SPI protocol in SPI Mode 3.

The PmodALS reports to the host board when the ADC081S021 is placed in normal mode by bringing the CS pin low, and delivers a single reading in 16 SCK clock cycles. The PmodALS requires the frequency of the SCK to be between 1 MHz and 4 MHz. **The bits of information, placed on the falling edge of the SCK and valid on the subsequent rising edge of SCK, consist of three leading zeroes, the eight bits of information with the MSB first, and four trailing zeroes.** An occasional 4th leading zero may be captured if CS goes low to initiate another data transfer before the rising edge SCLK. Any external power applied to the PmodALS must be within 2.7V and 5.25V; however, it is recommended that Pmod is operated at 3.3V.

I just drew the timing diagram based on the description from the reference above. Could you tell which mode this PMod sensor is operated at?



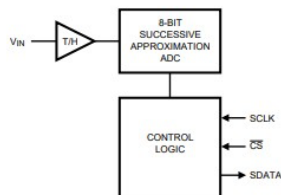
The circuit diagram of the ADC081S021 ADC:

8 Detailed Description

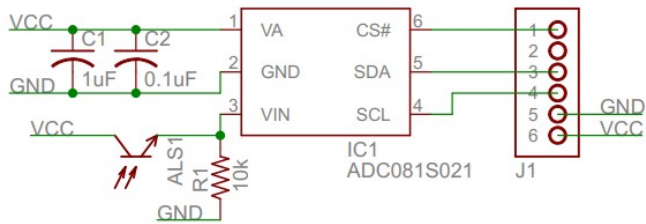
8.1 Overview

The ADC081S021 is a successive-approximation analog-to-digital converter designed around a charge-redistribution digital-to-analog converter core. Simplified schematics of the ADC081S021 in both track and hold modes are shown in Figure 19 and Figure 18, respectively. In Figure 19, the device is in track mode: switch SW1 connects the sampling capacitor to the input, and SW2 balances the comparator inputs. The device is in this state until CS is brought low, at which point the device moves to hold mode.

8.2 Functional Block Diagram



Schematic of the PMod module reported on Digilent website. Although the SDA/SCL labels are incorrect (should be SDO and SCK), we could still see the components that are palced on the module. The light sensor's output is pulled down to GND through a 10k resistor so in a dark environment, VIN will recieve 0 V. The ADC is an 8-bit SAR.

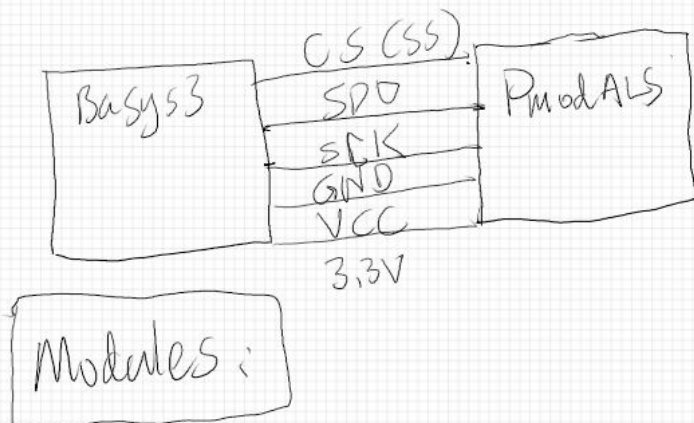


The ADC produces three leading zero bits on SDO, followed by eight data bits, most significant first. After the data bits, the ADC clocks out four trailing zeros. If CS goes low before the rising edge of SCK, an additional (fourth) zero bit may be captured by the next falling edge of SCLK.

Let's don't worry about the 4th 0 among the leading 0's. We know that for sure the low 4 bits will be zeros so data starts from bit 5 ends at bit 12.

The system diagram follows:

PmodALS Ambient Light Sensor



SPI_Light Sensor.v

SPI_leader_receiver.v

BinarytoBCD.v

sevenseg_driver.v

decoder_7seg.v

The top module used positional association in these instantiations. It follows the following rules:

The parent signals are the signals in your top.

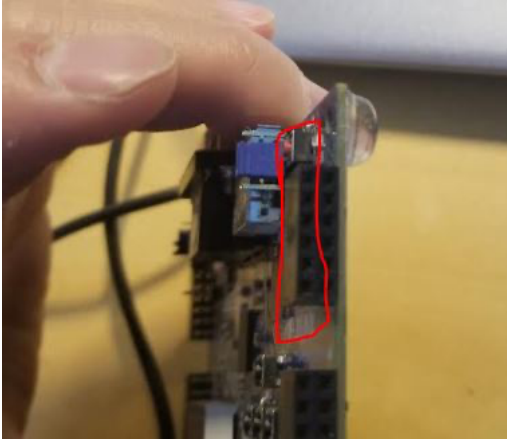
```
child_module instance_name (
    .child_port_name_A(parent_signal_A),
    .child_port_name_B(parent_signal_B),
    ...
);
```

Therefore, in the example code, is adding the leading four zeros necessary? Is using 'thos' necessary?

```
if(delay==0)
begin
    get<=1;
    state<=GETIN;
end
else
    delay<=delay-1'b1;
GETIN:
if(ready==1)
begin
    get<=0;
    lightdata<=data;
    delay<=2000*delaytime;
    state<=TRACK;
end
endcase
assign led={thos,huns,tens,ones};
SPI_leader_receiver #(16) spi(clk,sdo,get,data,sck,ss,busy,ready);
binarytoBCD_0 bcd({4'b0000,lightdata[12:5]},thos,huns,tens,ones);
sevenseg_driver_0 segdriver(clk,0,thos,huns,tens,ones,seg,an);
endmodule
```

```
module binarytoBCD(binary,thos,huns,tens,ones);
input [11:0] binary;
output reg [3:0] thos,huns,tens,ones;
reg [11:0] bcd_data=0;
always @ (binary)
begin
    bcd_data=binary;
    thos=bcd_data/1000;
    bcd_data=bcd_data%1000;
    huns=bcd_data/100;
    bcd_data=bcd_data%100;
    tens=bcd_data/10;
    ones=bcd_data%10;
end
endmodule
```

The PmodALS sensor is plugged in to the top row of the JB pins on the Basys 3 board.



Some changes must be made to the constraint file to make it work. Make those changes to the constraint file to make it work as demonstrated in the following video.



Yiyang Li



Watch on

I used a flash light to test the sensor. You can find that the maximum value is 255, which is the 8-bit binary value.

Tasks:

1. Repeat the tasks in Section 2 (50 points)
2. A smart light controller: the goal is to detect the room brightness and control the LED light output. The SPI transaction is periodically triggered to sample data from the Pmod ALS. The sensor returns an 8-bit value representing the current light level. PicoBlaze assembly code processes this 8-bit value and determine how many LEDs to be turned on. The brighter the ambient light, the less LEDs to be turned on. All LEDs should be on when it is dimmer than a certain value.(100 points)